

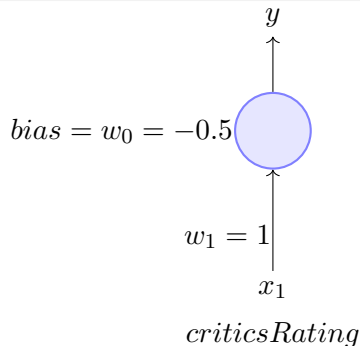
Module: Sigmoid Neuron

The story ahead ...

- Enough about boolean functions!
- What about arbitrary functions of the form $y = f(x)$ where $x \in \mathbb{R}^n$ (instead of $\{0,1\}^n$) and $y \in \mathbb{R}$ (instead of $\{0,1\}$) ?
- Can we have a network which can (approximately) represent such functions ?
- Before answering the above question we will have to first graduate from *perceptrons* to *sigmoidal neurons* ...

Recall

- A perceptron will fire if the weighted sum of its inputs is greater than the threshold ($-w_0$)



The thresholding logic used by a perceptron is very harsh !

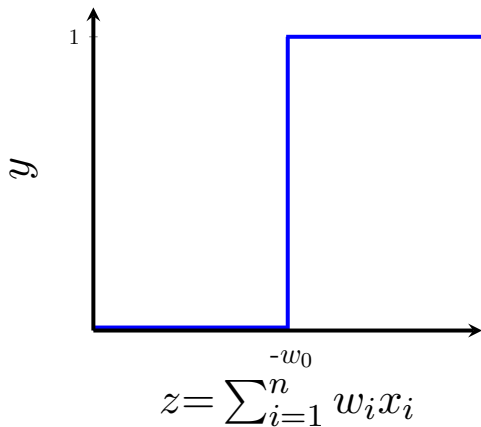
For example, let us return to our problem of deciding whether we will like or dislike a movie

Consider that we base our decision only on one input ($x_1 = criticsRating$ which lies between 0 and 1)

If the threshold is 0.5 ($w_0 = -0.5$) and $w_1 = 1$ then what would be the decision for a movie with $criticsRating = 0.51$? (like)

What about a movie with $criticsRating = 0.49$? (dislike)

It seems harsh that we would like a movie with rating 0.51 but not one with a rating of 0.49

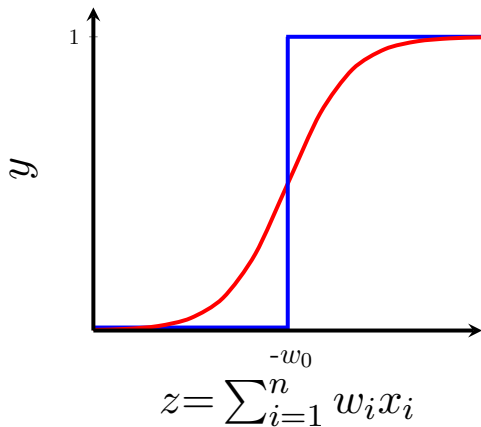


This behavior is not a characteristic of the specific problem we chose or the specific weight and threshold that we chose

It is a characteristic of the perceptron function itself which behaves like a step function

There will always be this sudden change in the decision (from 0 to 1) when $\sum_{i=1}^n w_i x_i$ crosses the threshold ($-w_0$)

For most real world applications we would expect a smoother decision function which gradually changes from 0 to 1



Introducing sigmoid neurons where the output function is much smoother than the step function

Here is one form of the sigmoid function called the logistic function

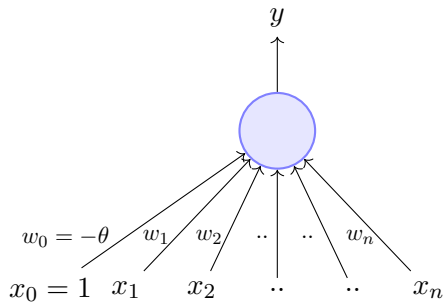
$$y = \frac{1}{1 + e^{-(w_0 + \sum_{i=1}^n w_i x_i)}}$$

We no longer see a sharp transition around the threshold $-w_0$

Also the output y is no longer binary but a real value between 0 and 1 which can be interpreted as a probability

Instead of a like/dislike decision we get the probability of liking the movie

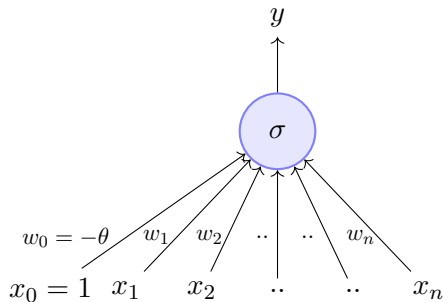
Perceptron



$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$

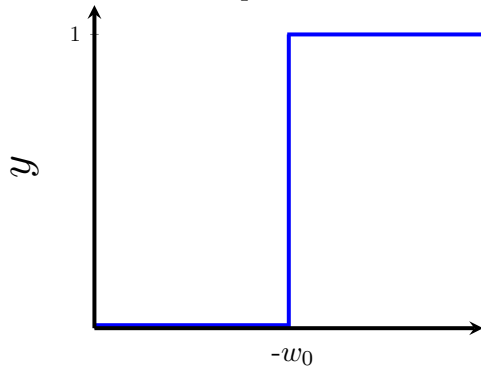
$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

Sigmoid (logistic) Neuron



$$y = \frac{1}{1 + e^{-(\sum_{i=0}^n w_i x_i)}}$$

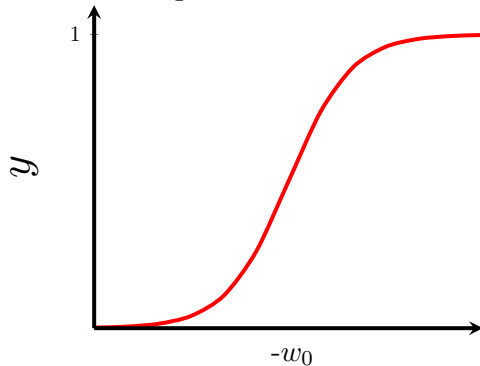
Perceptron



$$z = \sum_{i=1}^n w_i x_i$$

Not smooth, not continuous (at w_0), **not**
differentiable

Sigmoid Neuron

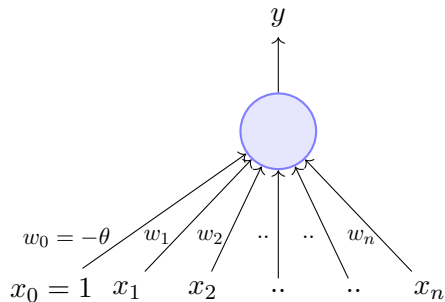


$$z = \sum_{i=1}^n w_i x_i$$

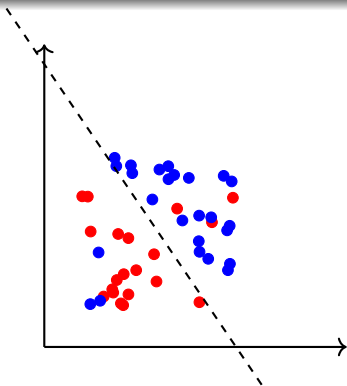
Smooth, continuous, **differentiable**

Module: A typical Supervised Machine Learning Setup

Sigmoid (logistic) Neuron



Well, just as we had an algorithm for learning the weights of a perceptron, we also need a way of learning the weights of a sigmoid neuron



Earlier we mentioned that a single perceptron cannot deal with this data because it is not linearly separable

What does “cannot deal with” mean?

What would happen if we use a perceptron model to classify this data ?

We would probably end up with a line like this ...

This line doesn't seem to be too bad

Sure, it misclassifies 3 blue points and 3 red points but we could live with this error in **most** real world applications

From now on, we will accept that it is hard to drive the error to 0 in most cases and will instead aim to reach the minimum possible error

This brings us to a typical machine learning setup which has the following components...

Data: $\{x_i, y_i\}_{i=1}^n$

Model: Our approximation of the relation between $\mathbf{x}$ and y . For example,

$$\hat{y} = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x})}}$$

or $\hat{y} = \mathbf{w}^T \mathbf{x}$

or $\hat{y} = \mathbf{x}^T \mathbf{W} \mathbf{x}$

or just about any function

Parameters: In all the above cases, w is a parameter which needs to be learned from the data

Learning algorithm: An algorithm for learning the parameters (w) of the model (for example, perceptron learning algorithm, gradient descent, etc.)

Objective/Loss/Error function: To guide the learning algorithm - the learning algorithm should aim to minimize the loss function

As an illustration, consider our movie example

Data: $\{x_i = \text{movie}, y_i = \text{like/dislike}\}_{i=1}^n$

Model: Our approximation of the relation between $\mathbf{x}$ and y (the probability of liking a movie).

$$\hat{y} = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x})}}$$

Parameter: $\mathbf{w}$

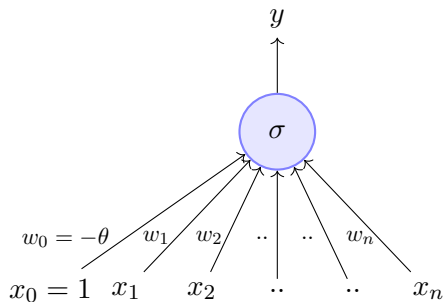
Learning algorithm: Gradient Descent [we will see soon]

Objective/Loss/Error function: One possibility is

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

The learning algorithm should aim to find a w which minimizes the above function (squared error between y and $\hat{y}$)

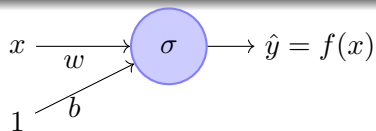
Module: Learning Parameters: (Infeasible) guess work



$$f(x) = \frac{1}{1+e^{-(w \cdot x + b)}}$$

Keeping this supervised ML setup in mind, we will now focus on this **model** and discuss an **algorithm** for learning the **parameters** of this model from some given **data** using an appropriate **objective function**

σ stands for the sigmoid function (logistic function in this case)



$$f(x) = \frac{1}{1+e^{-(w \cdot x + b)}}$$

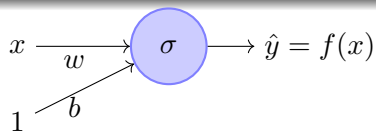
Keeping this supervised ML setup in mind, we will now focus on this **model** and discuss an **algorithm** for learning the **parameters** of this model from some given **data** using an appropriate **objective function**

σ stands for the sigmoid function (logistic function in this case)

For ease of explanation, we will consider a very simplified version of the model having just 1 input

Further to be consistent with the literature, from now on, we will refer to w_0 as b (bias)

Lastly, instead of considering the problem of predicting like/dislike, we will assume that we want to predict $criticsRating(y)$ given $imdbRating(x)$ (for no particular reason)



$$f(x) = \frac{1}{1+e^{-(w \cdot x + b)}}$$

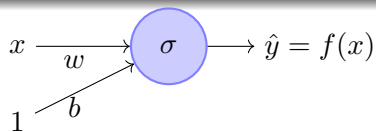
Input for training

$\{x_i, y_i\}_{i=1}^N \rightarrow N$ pairs of (x, y)

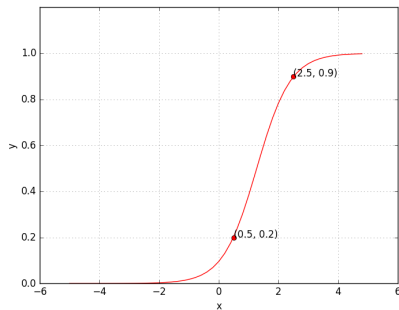
Training objective

Find w and b such that:

$$\underset{w, b}{\text{minimize}} \mathcal{L}(w, b) = \sum_{i=1}^N (y_i - f(x_i))^2$$



$$f(x) = \frac{1}{1+e^{-(w \cdot x + b)}}$$



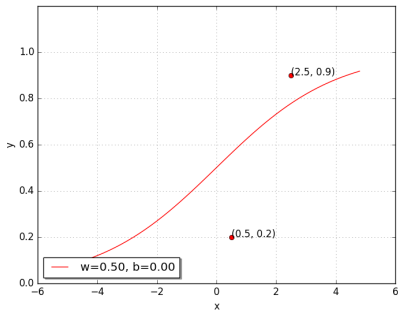
What does it mean to train the network?

- Suppose we train the network with $(x, y) = (0.5, 0.2)$ and $(2.5, 0.9)$
- At the end of training we expect to find w^* , b^* such that:
- $f(0.5) \rightarrow 0.2$ and $f(2.5) \rightarrow 0.9$

In other words...

- We hope to find a sigmoid function such that $(0.5, 0.2)$ and $(2.5, 0.9)$ lie on this sigmoid

Let us see this in more detail....



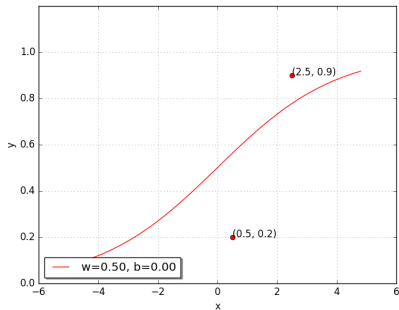
Can we try to find such a w^*, b^* manually

Let us try a random guess.. (say, $w = 0.5, b = 0$)

Clearly not good, but how bad is it ?

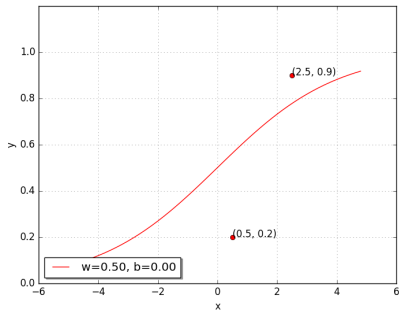
Let us revisit $\mathcal{L}(w, b)$ to see how bad it is ...

$$\sigma(x) = \frac{1}{1 + e^{-(wx+b)}}$$



$$\begin{aligned}\mathcal{L}(w, b) &= \frac{1}{2} * \sum_{i=1}^N (y_i - f(x_i))^2 \\ &= \frac{1}{2} * (y_1 - f(x_1))^2 + (y_2 - f(x_2))^2 \\ &= \frac{1}{2} * (0.9 - f(2.5))^2 + (0.2 - f(0.5))^2 \\ &= 0.073\end{aligned}$$

$$\sigma(x) = \frac{1}{1 + e^{-(wx+b)}}$$

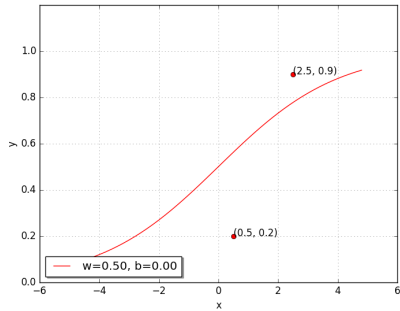


$$\begin{aligned}
 \mathcal{L}(w, b) &= \frac{1}{2} * \sum_{i=1}^N (y_i - f(x_i))^2 \\
 &= \frac{1}{2} * (y_1 - f(x_1))^2 + (y_2 - f(x_2))^2 \\
 &= \frac{1}{2} * (0.9 - f(2.5))^2 + (0.2 - f(0.5))^2 \\
 &= 0.073
 \end{aligned}$$

$$\sigma(x) = \frac{1}{1 + e^{-(wx+b)}}$$

We want $\mathcal{L}(w, b)$ to be as close to 0 as possible

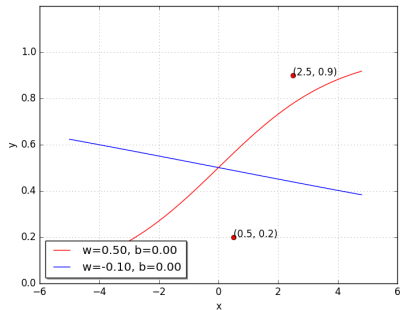
Let us try some other values of w , b



w	b	$\mathcal{L}(w, b)$
0.50	0.00	0.0730

$$\sigma(x) = \frac{1}{1 + e^{-(wx+b)}}$$

Let us try some other values of w , b

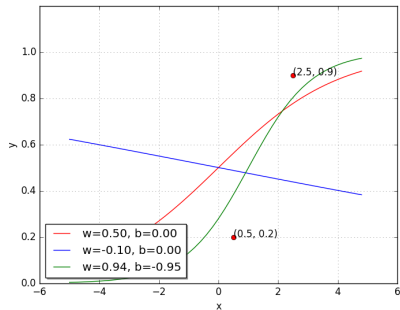


w	b	$\mathcal{L}(w, b)$
0.50	0.00	0.0730
-0.10	0.00	0.1481

$$\sigma(x) = \frac{1}{1 + e^{-(wx+b)}}$$

Oops!! this made things even worse...

Let us try some other values of w , b

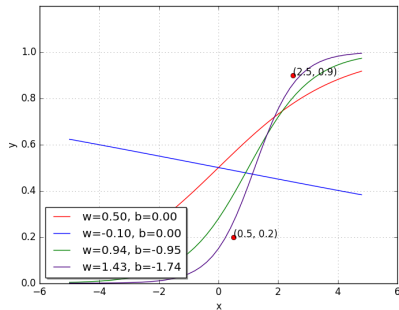


w	b	$\mathcal{L}(w, b)$
0.50	0.00	0.0730
-0.10	0.00	0.1481
0.94	-0.94	0.0214

$$\sigma(x) = \frac{1}{1 + e^{-(wx+b)}}$$

Perhaps it would help to push w and b in the other direction...

Let us try some other values of w , b

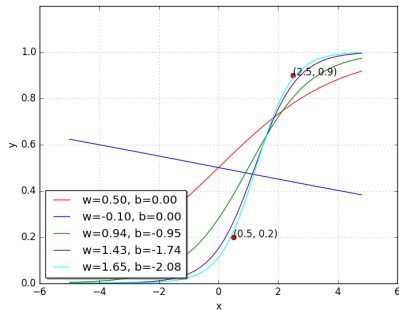


w	b	$\mathcal{L}(w, b)$
0.50	0.00	0.0730
-0.10	0.00	0.1481
0.94	-0.94	0.0214
1.42	-1.73	0.0028

$$\sigma(x) = \frac{1}{1 + e^{-(wx+b)}}$$

Let us keep going in this direction, *i.e.*, increase w and decrease b

Let us try some other values of w , b

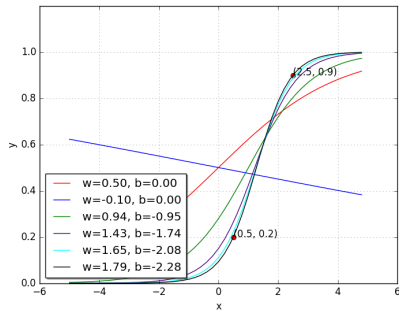


w	b	$\mathcal{L}(w, b)$
0.50	0.00	0.0730
-0.10	0.00	0.1481
0.94	-0.94	0.0214
1.42	-1.73	0.0028
1.65	-2.08	0.0003

$$\sigma(x) = \frac{1}{1 + e^{-(wx+b)}}$$

Let us keep going in this direction, *i.e.*, increase w and decrease b

Let us try some other values of w , b



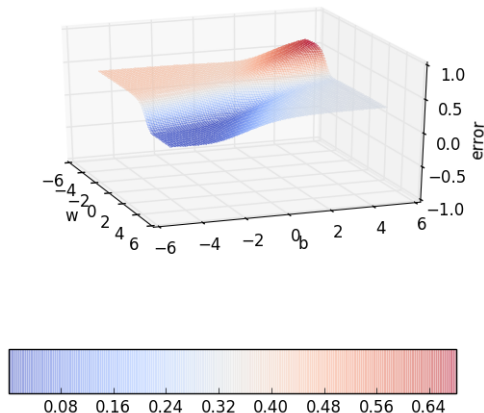
w	b	$\mathcal{L}(w, b)$
0.50	0.00	0.0730
-0.10	0.00	0.1481
0.94	-0.94	0.0214
1.42	-1.73	0.0028
1.65	-2.08	0.0003
1.78	-2.27	0.0000

$$\sigma(x) = \frac{1}{1 + e^{-(wx+b)}}$$

With some guess work and intuition we were able to find the right values for w and b

Let us look at something better than our “guess work” algorithm....

Random search on error surface



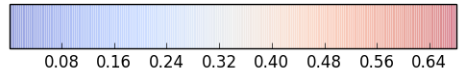
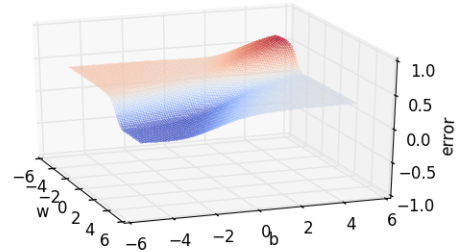
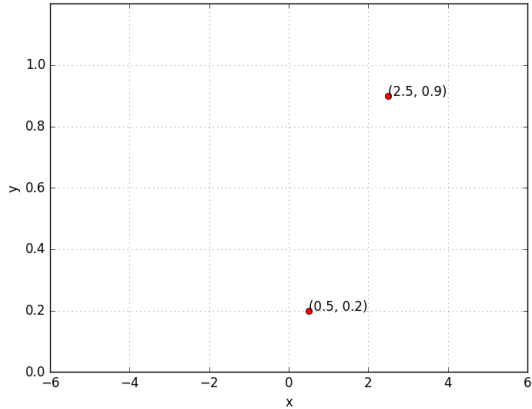
Since we have only 2 points and 2 parameters (w, b) we can easily plot $\mathcal{L}(w, b)$ for different values of (w, b) and pick the one where $\mathcal{L}(w, b)$ is minimum

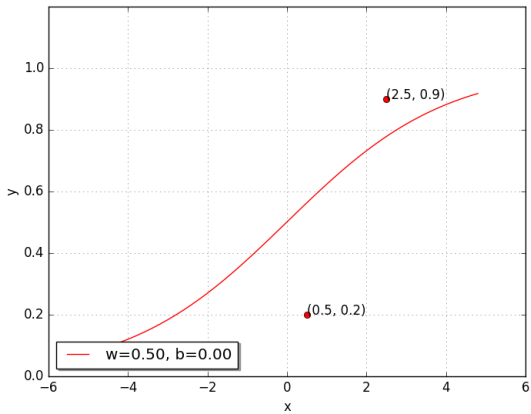
But of course this becomes intractable once you have many more data points and many more parameters !!

Further, even here we have plotted the error surface only for a small range of (w, b) [from $(-6, 6)$ and not from $(-\infty, \infty)$]

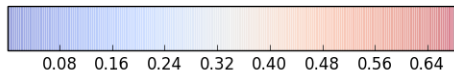
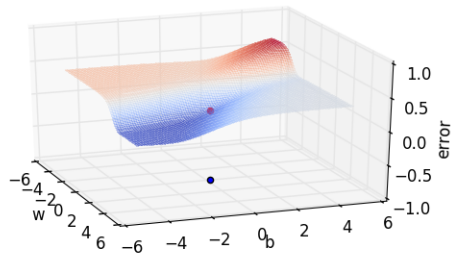
*Let us look at the geometric interpretation of our
“guess work” algorithm in terms of this error surface*

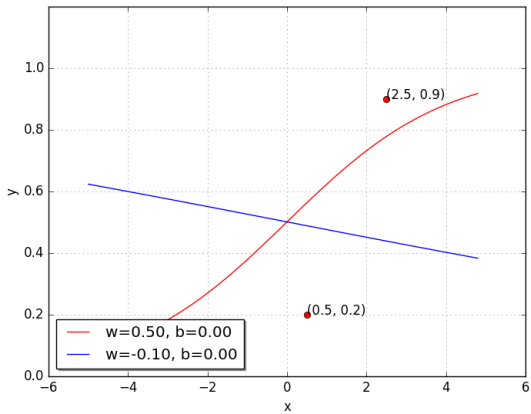
Random search on error surface



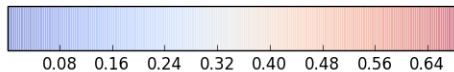
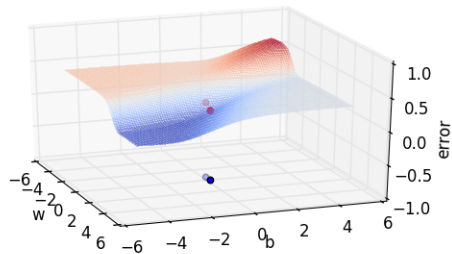


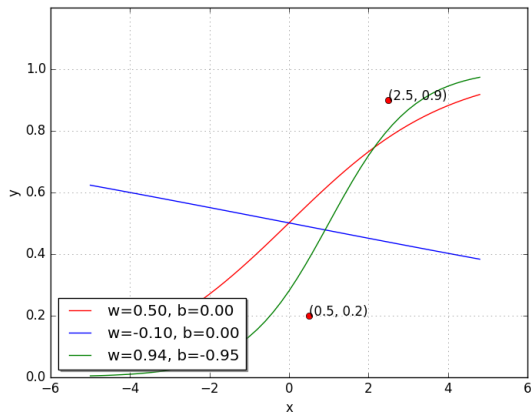
Random search on error surface



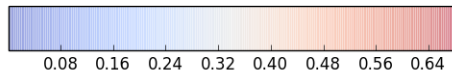
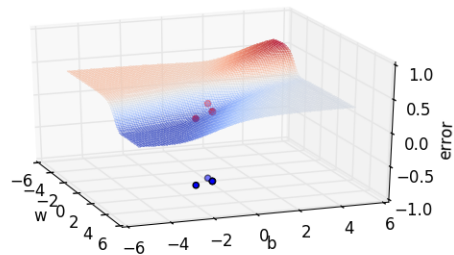


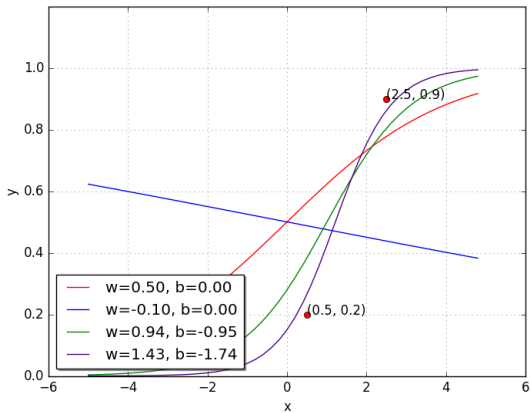
Random search on error surface



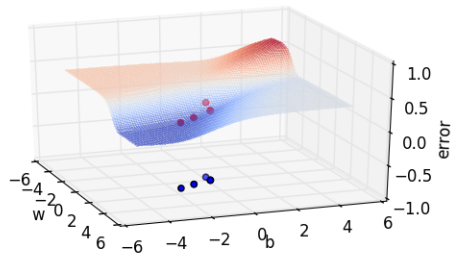


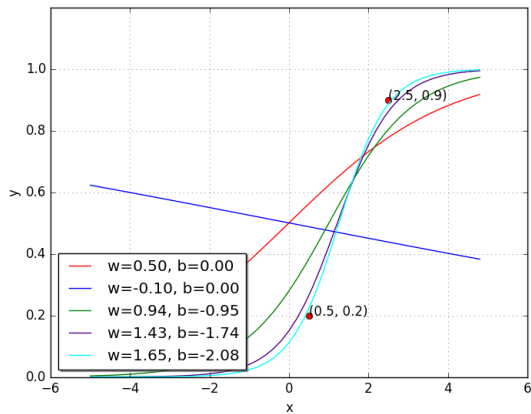
Random search on error surface



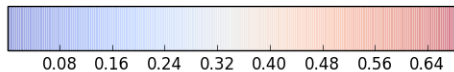
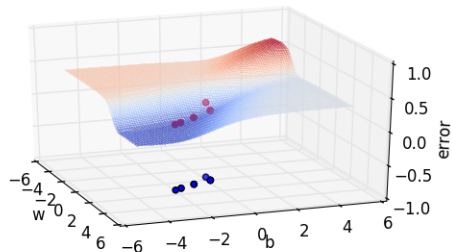


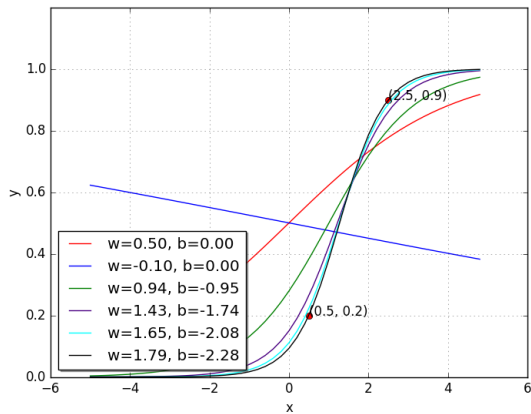
Random search on error surface



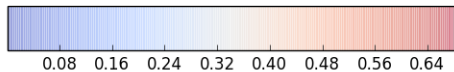
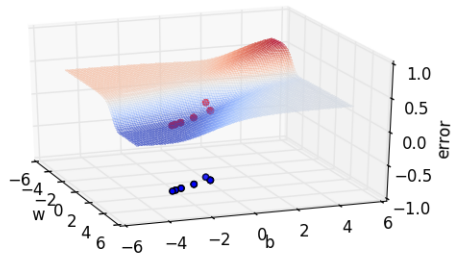


Random search on error surface





Random search on error surface

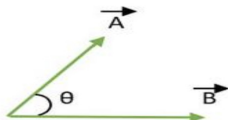


Module: Learning Parameters : Gradient Descent

*Now let us see if there is a more efficient and
principled way of doing this*

Goal

Find a better way of traversing the error surface so that we can reach the minimum value quickly without resorting to brute force search!



Two vector A and B

Dot product of A and B is given by,

$$\vec{A} \cdot \vec{B} = |A| |B| \cos\theta.$$

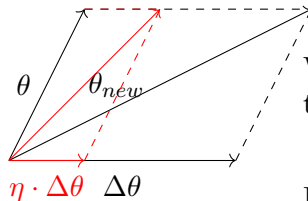
Error surface: The error surface is the 3-D surface that shows how the loss (error) changes as the model parameters change.

vector of parameters,
say, randomly initial-
ized

→ $\theta = [w, b]$

→ $\Delta\theta = [\Delta w, \Delta b]$

change in the
values of w, b



We moved in the direc-
tion of $\Delta\theta$

Let us be a bit conservat-
ive: move only by a small
amount η

$\theta_{new} = \theta + \eta \cdot \Delta\theta$

Question: What is the right $\Delta\theta$ to use
?

The answer comes from Taylor series

Taylor Series:

In mathematical analysis, the Taylor series or Taylor expansion of a function is an infinite sum of terms that are expressed in terms of the function's derivatives at a single point

- It approximates a function around a specific point or it finds the approximate value of a function

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f^{(3)}(a)}{3!}(x-a)^3 + \dots + \frac{f^{(n)}(a)}{n!}(x-a)^n + \dots$$

For ease of notation, let $\Delta\theta = u$, then from Taylor series, we have,

$$\begin{aligned}\mathcal{L}(\theta + \eta u) &= \mathcal{L}(\theta) + \eta * u^T \nabla_{\theta} \mathcal{L}(\theta) + \frac{\eta^2}{2!} * u^T \nabla^2 \mathcal{L}(\theta) u + \frac{\eta^3}{3!} * \dots + \frac{\eta^4}{4!} * \dots \\ &= \mathcal{L}(\theta) + \eta * u^T \nabla_{\theta} \mathcal{L}(\theta) \text{ [}\eta \text{ is typically small, so } \eta^2, \eta^3, \dots \rightarrow 0\text{]}\end{aligned}$$

Note that the move (ηu) would be favorable only if,

$$\mathcal{L}(\theta + \eta u) - \mathcal{L}(\theta) < 0 \text{ [i.e., if the new loss is less than the previous loss]}$$

This implies,

$$u^T \nabla_{\theta} \mathcal{L}(\theta) < 0$$

Okay, so we have,

$$u^T \nabla_{\theta} \mathcal{L}(\theta) < 0$$

But, what is the range of $u^T \nabla_{\theta} \mathcal{L}(\theta)$? Let us see....

Let β be the angle between u and $\nabla_{\theta} \mathcal{L}(\theta)$, then we know that,

$$-1 \leq \cos(\beta) = \frac{u^T \nabla_{\theta} \mathcal{L}(\theta)}{\|u\| * \|\nabla_{\theta} \mathcal{L}(\theta)\|} \leq 1$$

multiply throughout by $k = \|u\| * \|\nabla_{\theta} \mathcal{L}(\theta)\|$

$$-k \leq k * \cos(\beta) = u^T \nabla_{\theta} \mathcal{L}(\theta) \leq k$$

Thus, $\mathcal{L}(\theta + \eta u) - \mathcal{L}(\theta) = u^T \nabla_{\theta} \mathcal{L}(\theta) = k * \cos(\beta)$ will be most negative when $\cos(\beta) = -1$ i.e., when β is 180°

Gradient Descent Rule

- The direction u that we intend to move in should be at 180° w.r.t. the gradient
- In other words, move in a direction opposite to the gradient

Parameter Update Equations

$$w_{t+1} = w_t - \eta \nabla w_t$$

$$b_{t+1} = b_t - \eta \nabla b_t$$

$$\text{where, } \nabla w_t = \frac{\partial \mathcal{L}(w, b)}{\partial w} \text{ at } w = w_t, b = b_t, \nabla b = \frac{\partial \mathcal{L}(w, b)}{\partial b} \text{ at } w = w_t, b = b_t$$

So we now have a more principled way of moving in the w - b plane than our “guess work” algorithm

Let us create an algorithm from this rule ...

Algorithm: `gradient_descent()`

$t \leftarrow 0;$

$max_iterations \leftarrow 1000;$

while $t < max_iterations$ **do**

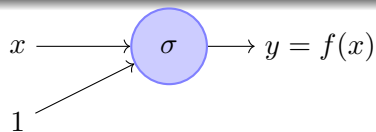
$w_{t+1} \leftarrow w_t - \eta \nabla w_t;$

$b_{t+1} \leftarrow b_t - \eta \nabla b_t;$

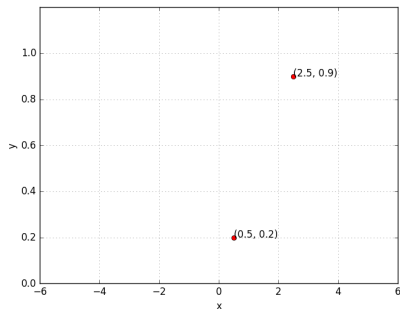
$t \leftarrow t + 1;$

end

To see this algorithm in practice let us first derive ∇w and ∇b for our toy neural network



$$f(x) = \frac{1}{1+e^{-(w \cdot x + b)}}$$



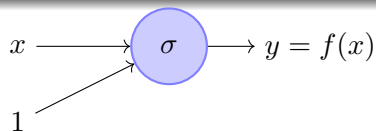
Let's assume there is only 1 point to fit (x, y)

$$\mathcal{L}(w, b) = \frac{1}{2} * (f(x) - y)^2$$

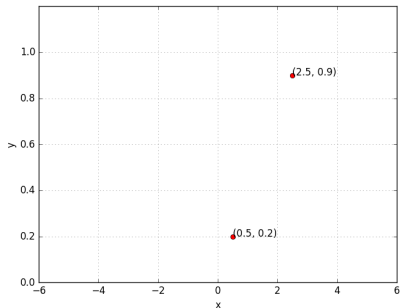
$$\nabla w = \frac{\partial \mathcal{L}(w, b)}{\partial w} = \frac{\partial}{\partial w} \left[\frac{1}{2} * (f(x) - y)^2 \right]$$

$$\begin{aligned}
\nabla w &= \frac{\partial}{\partial w} \left[\frac{1}{2} * (f(x) - y)^2 \right] \\
&= \frac{1}{2} * \left[2 * (f(x) - y) * \frac{\partial}{\partial w} (f(x) - y) \right] \\
&= (f(x) - y) * \frac{\partial}{\partial w} (f(x)) \\
&= (f(x) - y) * \frac{\partial}{\partial w} \left(\frac{1}{1 + e^{-(wx+b)}} \right) \\
&= (f(x) - y) * f(x) * (1 - f(x)) * x
\end{aligned}$$

$$\begin{aligned}
&\frac{\partial}{\partial w} \left(\frac{1}{1 + e^{-(wx+b)}} \right) \\
&= \frac{-1}{(1 + e^{-(wx+b)})^2} \frac{\partial}{\partial w} (e^{-(wx+b)}) \\
&= \frac{-1}{(1 + e^{-(wx+b)})^2} * (e^{-(wx+b)}) \frac{\partial}{\partial w} (-(wx + b)) \\
&= \frac{-1}{(1 + e^{-(wx+b)})} * \frac{e^{-(wx+b)}}{(1 + e^{-(wx+b)})} * (-x) \\
&= \frac{1}{(1 + e^{-(wx+b)})} * \frac{e^{-(wx+b)}}{(1 + e^{-(wx+b)})} * (x) \\
&= f(x) * (1 - f(x)) * x
\end{aligned}$$



$$f(x) = \frac{1}{1+e^{-(w \cdot x + b)}}$$



So if there is only 1 point (x, y) , we have,

$$\nabla w = (f(x) - y) * f(x) * (1 - f(x)) * x$$

For two points,

$$\nabla w = \sum_{i=1}^2 (f(x_i) - y_i) * f(x_i) * (1 - f(x_i)) * x_i$$

$$\nabla b = \sum_{i=1}^2 (f(x_i) - y_i) * f(x_i) * (1 - f(x_i))$$

```
X = [0.5, 2.5]  
Y = [0.2, 0.9]
```

```
X = [0.5, 2.5]  
Y = [0.2, 0.9]  
  
def f(w,b,x) : #sigmoid with parameters w,b  
    return 1.0 / (1.0 + np.exp(-(w*x + b)))
```

```
X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err
```

```

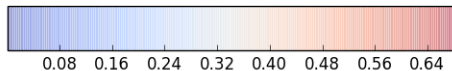
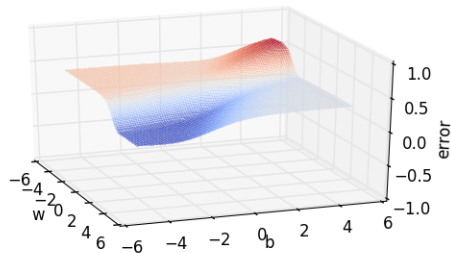
X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

```

Random search on error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

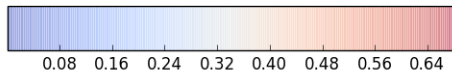
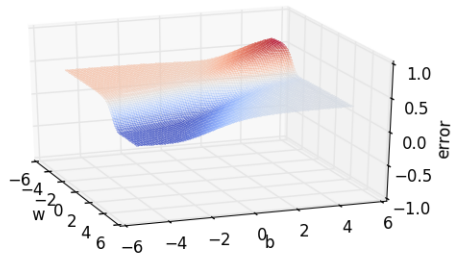
def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

```

Random search on error surface




```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

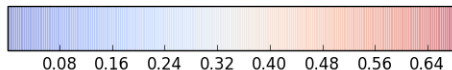
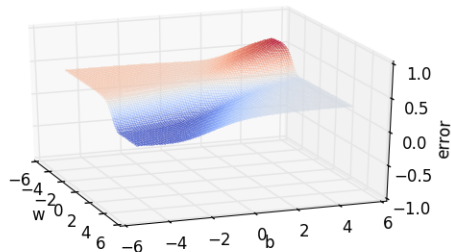
def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

```

Random search on error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

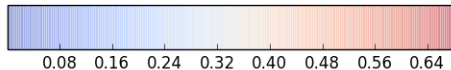
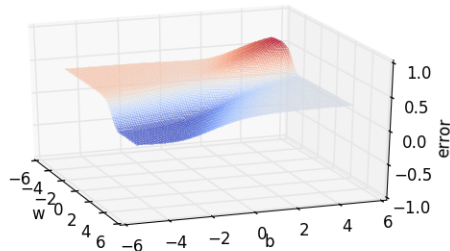
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Random search on error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

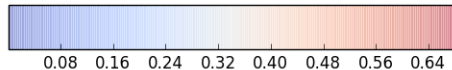
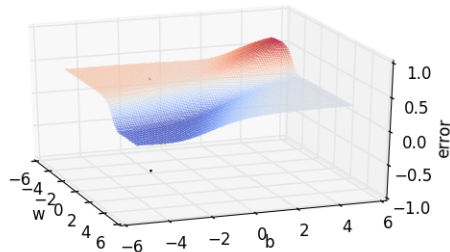
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

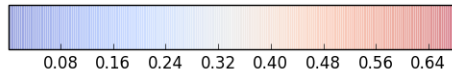
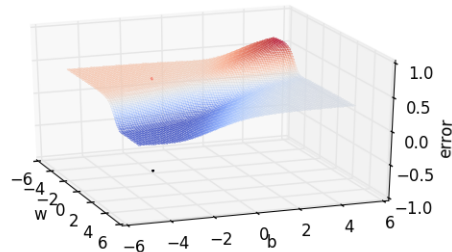
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

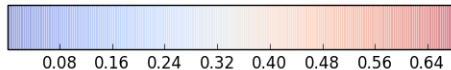
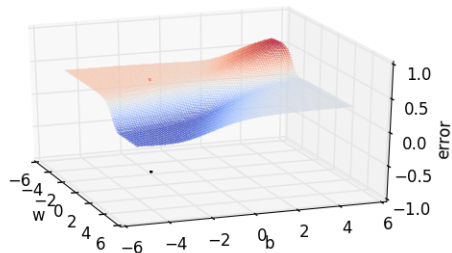
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

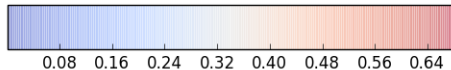
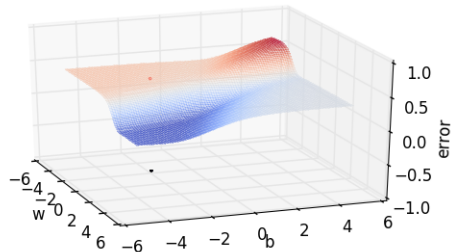
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

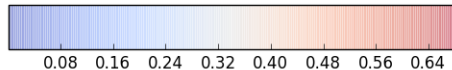
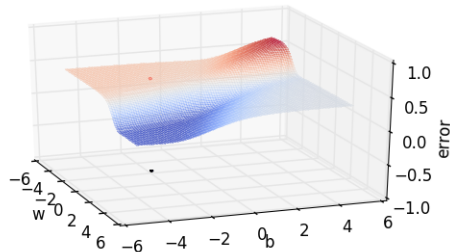
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

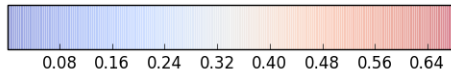
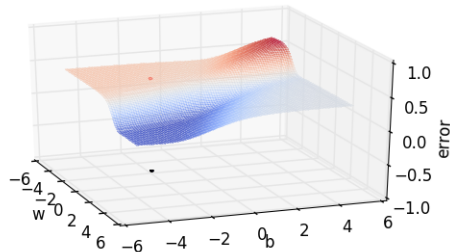
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface




```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

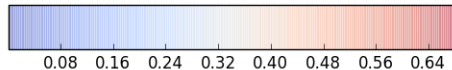
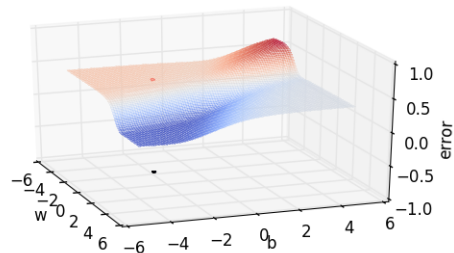
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

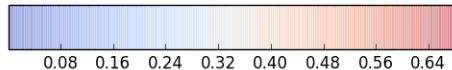
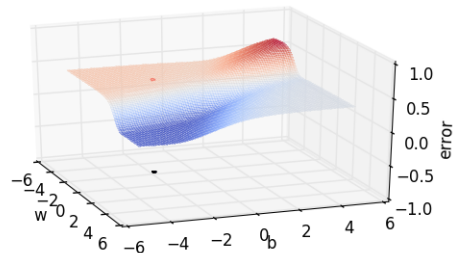
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

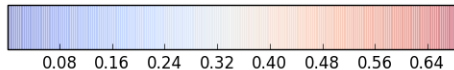
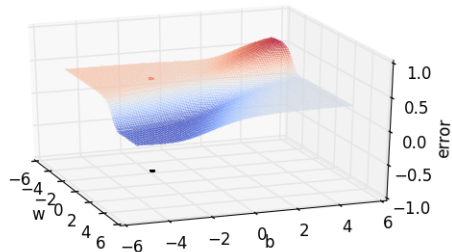
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

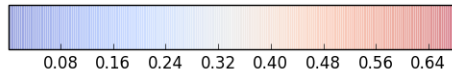
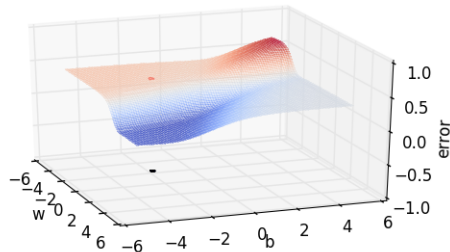
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

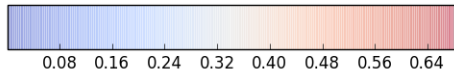
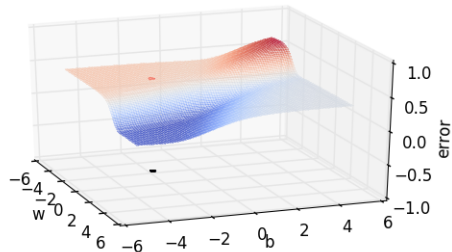
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

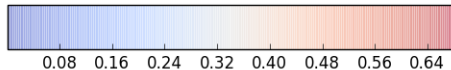
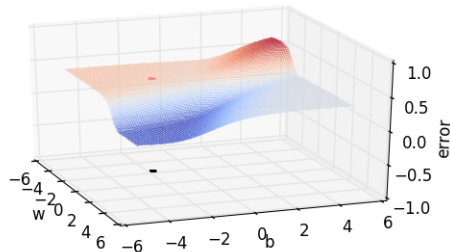
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

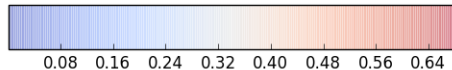
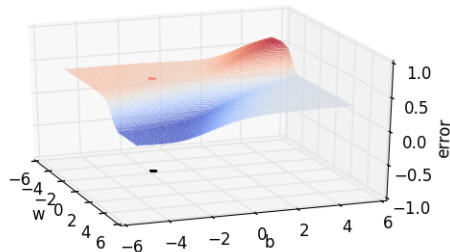
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

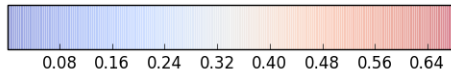
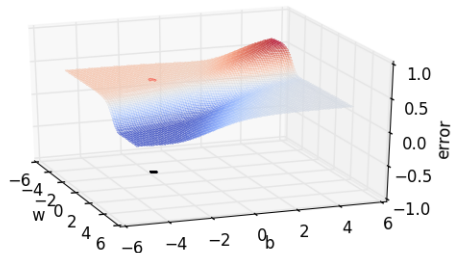
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface




```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

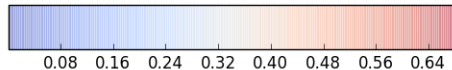
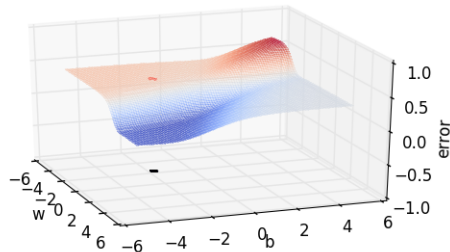
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

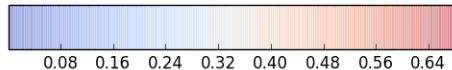
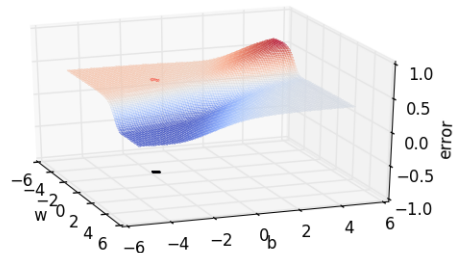
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

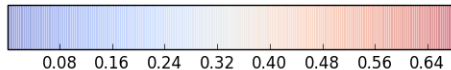
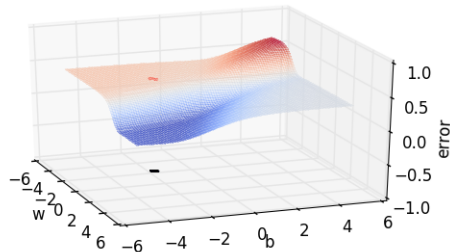
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

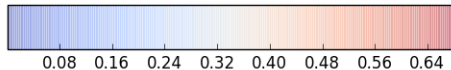
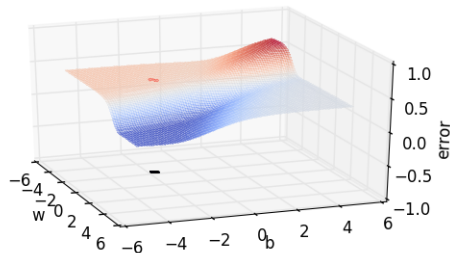
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

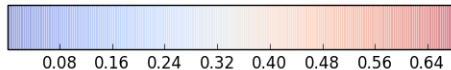
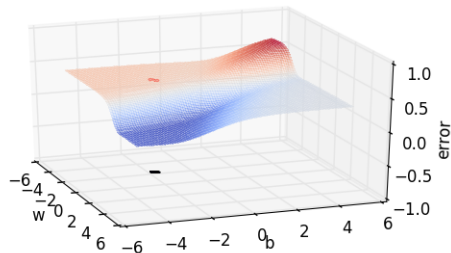
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

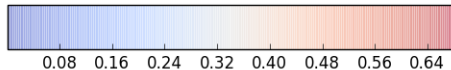
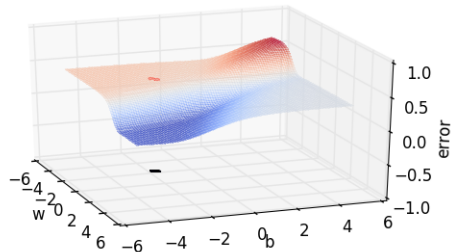
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

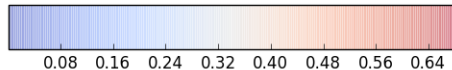
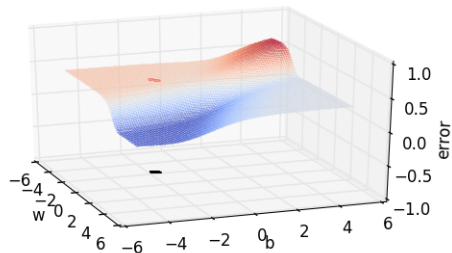
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

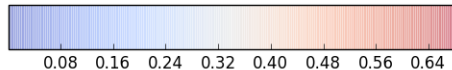
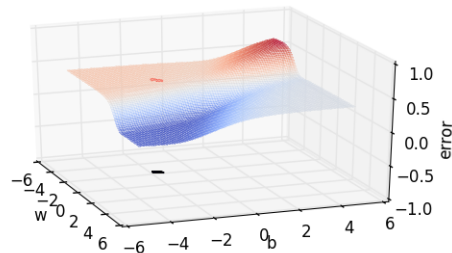
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface




```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

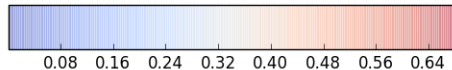
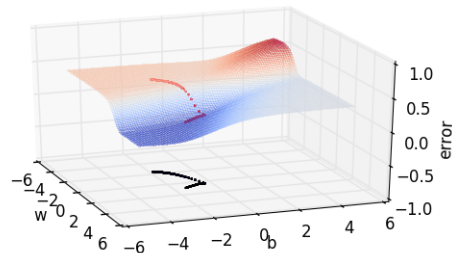
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

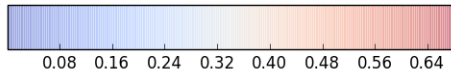
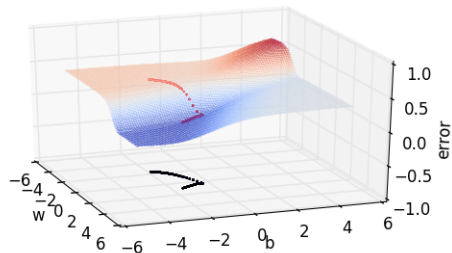
Gradient descent on the error surface



```
X = [0.5, 2.5]  
Y = [0.2, 0.9]
```

```
def f(w,b,x) : #sigmoid with parameters w,b  
    return 1.0 / (1.0 + np.exp(-(w*x + b)))  
  
def error (w, b) :  
    err = 0.0  
    for x,y in zip(X,Y) :  
        fx = f(w,b,x)  
        err += 0.5 * (fx - y) ** 2  
    return err  
  
def grad_b(w,b,x,y) :  
    fx = f(w,b,x)  
    return (fx - y) * fx * (1 - fx)  
  
def grad_w(w,b,x,y) :  
    fx = f(w,b,x)  
    return (fx - y) * fx * (1 - fx) * x  
  
def do_gradient_descent() :  
    w, b, eta, max_epochs = -2, -2, 1.0, 1000  
    for i in range(max_epochs) :  
        dw, db = 0, 0  
        for x,y in zip(X, Y) :  
            dw += grad_w(w, b, x, y)  
            db += grad_b(w, b, x, y)  
        w = w - eta * dw  
        b = b - eta * db
```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

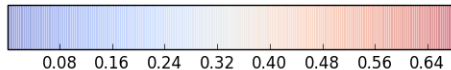
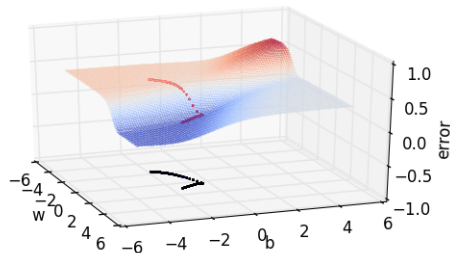
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

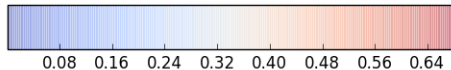
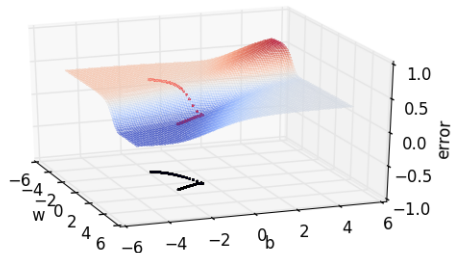
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

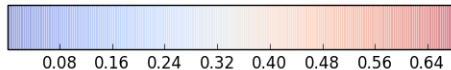
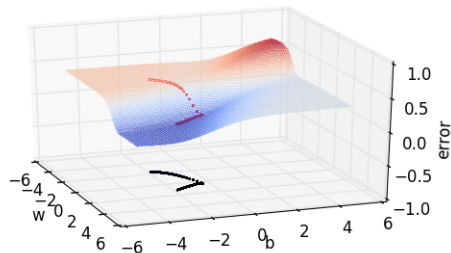
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



```

X = [0.5, 2.5]
Y = [0.2, 0.9]

def f(w,b,x) : #sigmoid with parameters w,b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error (w, b) :
    err = 0.0
    for x,y in zip(X,Y) :
        fx = f(w,b,x)
        err += 0.5 * (fx - y) ** 2
    return err

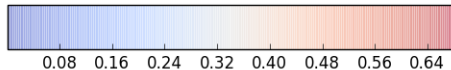
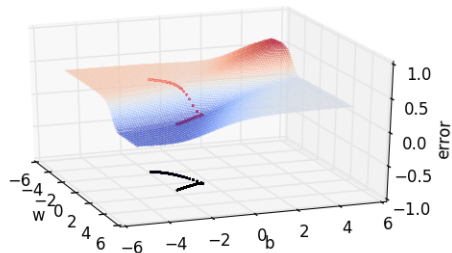
def grad_b(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w,b,x,y) :
    fx = f(w,b,x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent() :
    w, b, eta, max_epochs = -2, -2, 1.0, 1000
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db

```

Gradient descent on the error surface



We have studied an algorithm for learning the parameters of a sigmoid neuron

Module: Representation Power of a Multilayer Network of Sigmoid Neurons

Representation power of a multilayer network of perceptrons

A multilayer network of perceptrons with a single hidden layer can be used to represent any boolean function precisely (no errors)

Representation power of a multilayer network of sigmoid neurons

A multilayer network of neurons with a single hidden layer can be used to approximate any continuous function to any desired precision

In other words, there is a guarantee that for any function $f(x) : \mathbb{R}^n \rightarrow \mathbb{R}^m$, we can always find a neural network (with 1 hidden layer containing enough neurons) whose output $g(x)$ satisfies $|g(x) - f(x)| < \epsilon$!!